About this PDF

Built only from: **Theory of Computation (Automata and Formal Languages)** — https://nptel.ac.in/courses/106104028

Licence: CC BY-SA 4.0. Accessed .

Covers 10 of 10 syllabus sub-topics of this unit.

Need for automata theory

Automata theory studies abstract computational devices.

Definition

Automata theory studies abstract computational devices.

Explanation

It is crucial for compiler lexical analysis, hardware/circuit verification, protocol engineering, string pattern matching (KMP), and formal limits of computation.

Key points

- Automata theory studies abstract computational devices.
- It is crucial for compiler lexical analysis.
- It is crucial for hardware/circuit verification.
- It is crucial for protocol engineering.
- It is crucial for string pattern matching (KMP).

Introduction to formal proof

Formal proof is a systematic method for establishing the truth of mathematical statements using logical reasoning.

Definition

Formal proof refers to the use of deductive proof (modus ponens), mathematical and structural induction (base step, hypothesis, inductive step), proof by contradiction, and counterexample as methods to demonstrate the validity of statements.

Explanation

Deductive proof applies logical inference rules such as modus ponens to derive conclusions from premises. Mathematical and structural induction involve a base step, an inductive hypothesis, and an inductive step to prove statements for all natural numbers or recursively defined structures. Proof by contradiction shows a statement false by deriving a contradiction, while counterexample disproves a universal claim by providing a specific instance.

Key points

- Deductive proof uses logical inference rules like modus ponens.
- Induction requires a base case, hypothesis, and inductive step.
- Proof by contradiction demonstrates falsity by deriving inconsistency.

- Counterexample disproves universal statements.

Finite Automata (FA)

Definition

A finite automaton is a 5-tuple M = (Q, Sigma, delta, q0, F) where Q is a finite set of states, Sigma is an alphabet, delta is a transition function, q0 is the start state, and F is the set of accepting states. The automaton accepts a language L(M).

Explanation

Finite automata process input strings symbol by symbol. In a deterministic finite automaton (DFA), delta maps each pair (state, symbol) to exactly one next state, and there are no epsilon-transitions; the extended transition delta processes a whole word in linear time. In a nondeterministic finite automaton (NFA), delta maps each pair to a set of possible next states, allowing zero, one, or many transitions for a symbol; a word is accepted if any computation reaches an accepting state. Automata may also contain epsilon-transitions, which allow spontaneous moves without consuming input; the epsilon-closure ECLOSE(q) gives all states reachable from q via epsilon-moves, and such automata are computationally equivalent to standard DFAs. NFAs and DFAs are equivalent in expressive power; an n-state NFA can be converted to a DFA with at most 2^n states using subset construction. NFAs with epsilon-moves can be transformed into equivalent standard NFAs by first taking epsilon-closures before and after each symbol transition. The subset construction for converting an NFA to a DFA starts with ECLOSE(q0), then for each symbol a in Sigma, computes the set of states reachable from the current subset via a, and marks a subset as accepting if it contains any accepting state of the original NFA. DFAs can be minimized by marking distinguishable state pairs (accepting vs non-accepting) and iteratively propagating distinctions; indistinguishable states are merged into equivalence classes, yielding a minimal DFA.

Algorithm / Steps

1. **Subset Construction (NFA → DFA)** Initialize the start subset as ECLOSE(q0). For each symbol a in Sigma, compute the set of states reachable from the current subset via a, then take the epsilon-closure of that set. Add the resulting subset as a new DFA state if not already present. Mark a DFA state as accepting if it contains any accepting state of the NFA.
2. **Conversion of epsilon-NFA to NFA** For each state q and symbol a, compute delta'(q, a) = ECLOSE(delta(ECLOSE(q), a)). Set the start state q0 in F' if ECLOSE(q0) intersects F.
3. **DFA Minimization (Table-Filling Algorithm)** Mark all pairs of states where one is accepting and the other is not. Iteratively mark pairs (p, q) if there exists a symbol a such that (delta(p, a), delta(q, a)) is already marked. After convergence, merge all unmarked pairs into equivalence classes to form the minimal DFA.

Key points

- A finite automaton is defined by a 5-tuple (Q, Sigma, delta, q0, F).

- epsilon-transitions are handled via epsilon-closure and do not change computational power.
- DFAs can be minimized by distinguishing state pairs and merging indistinguishable ones.

Deterministic Finite Automata (DFA)

Definition

A deterministic finite automaton (DFA) is a 5-tuple M = (Q, Sigma, delta, q_0, F) where Q is a finite set of states, Sigma is the input alphabet, delta : Q x Sigma -> Q is a deterministic transition function, q_0 in Q is the start state, and F subset-of Q is the set of accepting states. The DFA has exactly one transition for each state and input symbol and contains no epsilon-transitions.

Explanation

In a DFA, the transition function delta is deterministic: for every state q in Q and symbol a in Sigma there is exactly one state delta(q,a) in Q. The extended transition function delta_hat(q, w) for a word w in Sigma^* is defined recursively: delta_hat(q, epsilon) = q and delta_hat(q, wa) = delta(delta_hat(q, w), a). Processing an input word of length |w| takes O(|w|) time because each symbol is examined once.

Algorithm / Steps

1. Compute ECLOSE(q_0) to form the initial DFA state.
2. For each DFA state T subset-of Q and each symbol a in Sigma, compute U = ECLOSE(delta(T, a)) where delta(T, a) = {delta(q, a) | q in T}.
3. Add U as a new DFA state if it is not already present.
4. Mark a DFA state as accepting if it contains any accepting state of the NFA.
5. Repeat until no new states are added. An n-state NFA yields at most 2^n states in the resulting DFA.

Key points

- A DFA is a 5-tuple with a deterministic transition function and no epsilon-transitions.
- The extended transition function processes input words in linear time.
- Subset construction converts an NFA to an equivalent DFA by tracking reachable state subsets.
- The resulting DFA can have up to 2^n states for an n-state NFA.

Non-deterministic Finite Automata (NFA)

Definition

An NFA is a finite automaton in which the transition function maps a state and an input symbol to a set of possible next states. The machine accepts a word if the extended transition function applied to the start state and the word yields a set of states that intersects the set of accepting states.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Explanation

In an NFA, for a given state and symbol, there may be zero, one, or multiple transitions. The machine accepts a word if any computation path leads to an accepting state. Some NFAs allow epsilon transitions, which enable spontaneous moves without consuming input; the epsilon-closure of a state is the set of states reachable via epsilon-paths. NFAs with epsilon-moves can be transformed into equivalent standard NFAs by computing epsilon-closures before and after each symbol transition. Every NFA can be converted to an equivalent DFA using the subset construction, producing at most 2^n states for an n-state NFA.

Algorithm / Steps

1. Compute the epsilon-closure of the start state to form the initial DFA state.
2. For each DFA state (a subset of NFA states) and each input symbol, compute the set of NFA states reachable from any state in the subset via that symbol, then take the epsilon-closure of that set to form a new DFA state.
3. Mark a DFA state as accepting if it contains any NFA accepting state.
4. Repeat until no new DFA states are generated.

Key points

- The transition function maps to subsets of states.
- Acceptance depends on the intersection of the reachable set with the accepting set.
- Epsilon transitions are handled via epsilon-closure.
- NFAs are equivalent to DFAs through subset construction.
- Subset construction may produce up to 2^n states for an n-state NFA.

Equivalence between NFA and DFA Finite Automata with Epsilon transitions

Definition

An NFA with epsilon transitions is a finite automaton that may change state spontaneously without consuming input symbols. It is computationally equivalent to a DFA.

Explanation

The equivalence is achieved by computing epsilon-closures of states. The epsilon-closure ECLOSE(q) contains all states reachable from q via epsilon transitions. By using ECLOSE in the transition function, an NFA with epsilon moves can be transformed into an equivalent standard NFA, and then into a DFA via subset construction.

Algorithm / Steps

1. Compute ECLOSE(q0) for the start state q0.
2. For each symbol ainSigma, compute the set of states reachable from ECLOSE(q) by a, then take the epsilon-closure of that set to form delta'(q, a).

3. Use subset construction: each DFA state represents a subset of NFA states. Initialize the DFA start state with ECLOSE(q0).
4. For each DFA state T and each input symbol a, compute T' = ECLOSE(union_pinT delta(p, a)) and add transition T --a--> T'.
5. Mark a DFA state T as accepting if T contains any accepting state of the original NFA.
6. Repeat until no new DFA states are generated.

Key points

- An NFA with epsilon transitions can be converted to an equivalent standard NFA by incorporating epsilon-closures into the transition function.
- The subset construction algorithm transforms an NFA (with or without epsilon moves) into a DFA, potentially creating up to 2^n states.
- The epsilon-closure ECLOSE(q) is the set of states reachable from q via zero or more epsilon transitions.
- A DFA state is accepting if its corresponding subset contains at least one accepting state of the original NFA.
- The resulting DFA accepts the same language as the original NFA with epsilon transitions, proving computational equivalence.

Equivalence of NFA and DFA

Definition

Equivalence of NFA and DFA means that for every input string, the NFA accepts it if and only if the DFA accepts it.

Algorithm / Steps

1. Compute the epsilon-closure of the NFA start state, ECLOSE(q0).
2. Track all reachable subsets of NFA states; each subset becomes a DFA state.
3. Mark a DFA state as accepting if the corresponding subset contains any NFA accepting state.

Key points

- An NFA and a DFA are equivalent if they accept the same language.
- Epsilon transitions are handled by the epsilon-closure operation.
- The resulting DFA accepts exactly the language of the original NFA.

Equivalence of NFAs with and without ε-moves

Definition

An NFA with epsilon-moves is a finite automaton that allows spontaneous state changes without consuming input symbols. It is formally described by the transition function delta that may include epsilon as a possible input symbol, enabling transitions that do not read any character from the input string.

Explanation

The equivalence between NFAs with epsilon-moves and standard NFAs (without epsilon-moves) is achieved by constructing an equivalent standard NFA whose transitions incorporate the effect of epsilon-moves through the epsilon-closure operation. The epsilon-closure of a state q, denoted ECLOSE(q), is the set of states reachable from q by following any number of epsilon-transitions. By redefining the transition function to use these closures, the resulting automaton accepts the same language as the original epsilon-NFA.

Algorithm / Steps

1. For each state q and input symbol a, compute the set of states reachable from q by first taking the epsilon-closure of q, then following all a-transitions, and finally taking the epsilon-closure of the resulting states: delta'(q, a) = ECLOSE(delta(ECLOSE(q), a)).
2. Define the start state of the new NFA to be the epsilon-closure of the original start state q0.
3. Mark a state q' as accepting in the new NFA if the epsilon-closure of q0 contains at least one accepting state of the original epsilon-NFA.

Key points

- epsilon-moves allow spontaneous transitions without consuming input.
- The epsilon-closure of a state captures all states reachable via epsilon-transitions.
- An equivalent standard NFA can be constructed by redefining transitions using epsilon-closures.
- The start state and accepting states of the new NFA are determined from the epsilon-closure of the original start state.

Conversion of NFA into DFA

Definition

Conversion of an NFA into a DFA is achieved by the subset construction method, which builds a DFA whose states are subsets of the NFA's states. The construction starts with the epsilon-closure of the NFA's start state and then, for each input symbol, computes the reachable subsets. Final DFA states are those subsets that contain at least one NFA accepting state.

Algorithm / Steps

1. Initialize the DFA start state as ECLOSE(q0) of the NFA.
2. For each DFA state T subset-of Q and each input symbol a in Sigma, compute the set of NFA states reachable from any state in T on a, then take the epsilon-closure of that set; this becomes the DFA transition delta(T, a).
3. Mark a DFA state T as accepting if T contains at least one accepting state of the NFA.

Key points

- Epsilon transitions are incorporated via ECLOSE, ensuring equivalence to the original NFA.
- The resulting DFA accepts the same language as the NFA.

Minimization of DFAs.

Definition

Minimization of DFAs reduces the number of states while preserving the accepted language, using the Myhill-Nerode theorem and the Table-Filling Algorithm.

Explanation

The process marks distinguishable state pairs (accepting vs non-accepting), propagates distinctions iteratively, and merges indistinguishable states into equivalence classes.

Algorithm / Steps

1. Mark all state pairs where one state is accepting and the other is non-accepting.
2. Propagate distinctions iteratively: if a pair leads to a marked pair on some input symbol, mark it.
3. Merge all indistinguishable states into equivalence classes to form the minimized DFA.

Key points

- Minimization uses the Myhill-Nerode theorem and the Table-Filling Algorithm.
- Distinguishable state pairs are marked and distinctions propagated.
- Indistinguishable states are merged into equivalence classes.

Source and licence

This PDF is adapted from **Theory of Computation (Automata and Formal Languages)** (https://nptel.ac.in/courses/106104028), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).